

數位人頭像即時唇形同步 與逐字標示字幕

新功能技術分析報告

Avatar Lip-Sync & Karaoke-style Word Highlighting

專案：cooleng_with_avatar_sync（Cool English 英檢口說練習）

技術棧：SvelteKit 5 · TypeScript · Azure Speech SDK · OpenAI

報告日期：2026-06-19

目錄

目錄.....	2
1. 執行摘要	4
2. 新功能目標的理解與拆解.....	6
2.1 影片所展示的功能本質	6
2.2 將需求拆解為四個可獨立攻克的子問題.....	6
3. 現有系統技術分析	8
3.1 技術棧與相依	8
3.2 現有 TTS 流程（關鍵整合點）	8
3.3 現有對話/朗讀流程.....	9
4. 技術挑戰深入分析	10
4.1 端到端延遲與串流分句	10
4.2 音訊、嘴型、字幕三軌時間對齊.....	10
4.3 瀏覽器渲染效能（若採 3D 路線）	10
4.4 GPU 後端與串流（若採 2D 神經頭像路線）	10
4.5 頭像資產與授權	11
5. 學術方法與開源方案評估	12
5.1 方法 A（首選）：Viseme/Blendshape 驅動 3D 頭像.....	12
5.2 方法 B（備選）：MuseTalk 即時 2D 神經對嘴（照片級寫實）	12
5.3 方法 C（備選）：Wav2Lip 基準 / NVIDIA Audio2Face 3D.....	13
5.4 三方案綜合比較	13
6. 字幕逐字標示（單字級對齊）方案.....	15
6.1 首選：Azure WordBoundary 事件（零額外基礎設施）	15
6.2 備選：強制對齊（僅在對「既有音檔」對齊時才需要）	15
7. 推薦整合架構與資料流	16
7.1 端到端資料流（首選方案 A）	16
7.2 建議新增/重構的程式元件.....	16
7.3 SSML 範例（取得 blendshape 與 word boundary）	16
8. 需額外新增的功能（分析過程中發現）	18
9. 分階段實作路線圖.....	19
10. 風險與緩解	20
11. 結論與建議	20
附錄 A：開源專案與授權清單	21

附錄 B：Azure Speech SDK 關鍵事件速查	21
------------------------------------	----

1. 執行摘要

本報告針對在現有「Cool English 英檢口說練習」系統中，新增一個會「邊說英文、邊對嘴、並在字幕中即時標示目前所唸單字」的數位人頭像（Avatar）功能，進行從需求理解到技術選型的完整分析。分析涵蓋四個層面：（1）功能目標的拆解、（2）現有系統的整合切入點、（3）需克服的技術挑戰、（4）具備 GitHub 開源實作之學術方法的評估與推薦。

核心發現（最關鍵的結論）

現有系統已經內建達成此功能所需的兩個關鍵訊號來源。專案已相依 Azure Speech SDK（`microsoft-cognitiveservices-speech-sdk`），而該 SDK 在語音合成（TTS）時原生提供：

① **visemeReceived 事件** —— 提供 ARKit 相容的 55 通道臉部 blendshape 動畫（60 FPS），可直接驅動 3D 頭像對嘴；

② **WordBoundary 事件** —— 提供每個單字的音訊起始時間與長度（以 100ns tick 計），可直接驅動字幕逐字標示，無需另外導入強制對齊（forced alignment）模型。

因此，最省力且與現有架構最契合的路線，是「Azure viseme/word-boundary 事件 + 瀏覽器端 3D 頭像渲染」。

推薦方案（擇一為主，另兩者為備選）：

1. 方法 A（首選）— **Viseme 驅動 3D 頭像**：以開源 JS 函式庫 met4citizen/TalkingHead（MIT）在瀏覽器中以 Three.js 渲染 Ready Player Me / glTF 3D 數位人，並以 Azure 的 viseme blendshape 串流驅動嘴型。完全在前端執行，無需 GPU 伺服器，與現有 Azure TTS 串接最自然。
2. 方法 B（備選，照片級寫實）— **MuseTalk 即時 2D 神經對嘴**：以伺服器 GPU 即時（30fps+）對一張真人臉照片做潛在空間 inpainting 產生對嘴影像，再以串流方式送回瀏覽器。寫實度最高，但需 GPU 後端與串流基礎設施。
3. 方法 C（備選，基準/3D 高品質）— **Wav2Lip 或 NVIDIA Audio2Face**：Wav2Lip 為經典 2D 對嘴基準（成熟、易重現，但授權限非商業）；Audio2Face 為 NVIDIA 於 2025 開源之即時 3D 臉部動畫（>60fps，需 NVIDIA GPU 後端）。

字幕逐字標示：一律推薦直接使用 Azure **WordBoundary** 事件，零額外基礎設施；若未來需對「非 Azure 合成」之既有音檔對齊，再退而採用 WhisperX 或 Montreal Forced Aligner。

2. 新功能目標的理解與拆解

2.1 影片所展示的功能本質

參考影片（YouTube：qPhbx_43MMM）展示的是一位真人講者一邊說英文、下方字幕一邊「對齊嘴唇即時呈現」，且會把「目前正在唸的單字」標示出來（類似卡拉 OK 的逐字高亮）。把這個視覺體驗對應到本專案的需求，實際上是兩件事的疊加：

- **視覺對嘴（Lip-Sync）**：一個頭像/數位人，其嘴型隨著語音逐音節變化，看起來像是「它在說這段話」。
- **逐字字幕標示（Word-level Highlighting）**：字幕文字依語音進度逐字點亮，使用者能清楚對應「聲音」與「文字」。

對一個語言學習產品而言，這兩者的結合具備明確教學價值：學習者能同時獲得 **聽覺（語音）**、**視覺（嘴型/發音口形）**、**文字（拼字）** 三模態的對齊輸入，有助於發音模仿與單字辨識。

2.2 將需求拆解為四個可獨立攻克的子問題

#	子問題	要解決的核心問題	本專案的最省力解
S1	頭像渲染（Avatar Rendering）	畫面上要有一個可動的數位人；其臉部需可被「嘴型參數」驅動。	瀏覽器端 3D glTF 頭像（TalkingHead.js），或伺服器端 2D 神經頭像（MuseTalk）。
S2	音視覺對嘴（A/V Lip-Sync）	把語音轉成隨時間變化的嘴型序列（viseme / blendshape），並與音訊播放對齊。	直接取用 Azure TTS 的 visemeReceived（ARKit 55-blendshape，60fps）。
S3	逐字字幕對齊與標示	取得每個單字對應的音訊時間區間，於播放時逐字高亮。	直接取用 Azure TTS 的 WordBoundary（audioOffset / duration / textOffset）。
S4	即時串流整合	LLM 逐 token 產生文字 → 分句 → TTS → 對嘴 + 字幕，需在對話流程中順暢串接且低延遲。	重構現有 Synthesizer，改為可掛載事件並由播放時鐘統一驅動 UI。

關鍵洞察：S2 與 S3 在本專案幾乎「免費」

多數團隊在做此功能時，最大的成本是 S2（如何從音訊推估嘴型）與 S3（如何做單字級時間對齊）。但因為本專案使用 Azure TTS 來「產生」語音，Azure 在合成當下就同時吐出 viseme 與 word-boundary 時間軸，等於把 S2、S3 的「對齊」問題在來源端解掉了。剩下真正要做的工程，集中在 S1（選一個頭像渲染法）與 S4（把事件接到 UI 的時間軸）。

3. 現有系統技術分析

3.1 技術棧與相依

層面	現況
前端框架	SvelteKit 2 + Svelte 5 (Runes : \$state / \$derived / \$effect) + TypeScript + Vite
UI	TailwindCSS 4 + Flowbite-Svelte + lucide-svelte 圖示
語音 (TTS/STT)	Azure Cognitive Services Speech SDK (microsoft-cognitiveservices-speech-sdk 1.43)
LLM	OpenAI SDK (評分與回饋 : /api/grader/*)
伺服器	SvelteKit server routes : Azure 權杖以 /api/speech/credentials 簽發 (token + region)
現有對話資料模型	src/lib/types.ts 的 Message 介面已預留 audio?、avatar? 欄位

3.2 現有 TTS 流程 (關鍵整合點)

語音合成封裝在 `src/lib/azure/speech/synthesizer.ts` 的 `Synthesizer` 類別。重點如下：

- 建立時使用 `SpeakerAudioDestination` 與 `AudioConfig.fromSpeakerOutput(player)`，亦即音訊由 SDK 直接「播到喇叭」。
- `synthesize()` 會把 LLM 輸出累積到 `partialSentenceBuffer`，偵測到句末標點 (.?!) 或逾時才送出整句給 `speakTextAsync()`——已具備「逐句」切分機制，對串流很有利。
- 已掛載 `synthesizing` (音訊分塊) 與 `synthesisCompleted` 事件，但尚未掛載 `visemeReceived` 與 `wordBoundary` 事件。

現有架構需要的關鍵重構

問題：目前 `SpeakerAudioDestination` 讓音訊「自動播放」，前端拿不到一個可查詢的播放時鐘，難以將嘴型/字幕精準對齊到「現在播到第幾毫秒」。

方向：將合成輸出改為可控的音訊串流（例如合成到 `ArrayBuffer/MediaSource` 或改用 `SpeakerAudioDestination` 的 `onAudioStart/位置回呼`），讓 `viseme` 與 `word-boundary` 的 `tick` 能對齊到同一個播放時鐘，UI 才能逐字、逐 frame 精準驅動。此為 S4 的核心工程。

3.3 現有對話/朗讀流程

以 `src/routes/question-answering/+page.svelte` 為例：onMount 建立 `Synthesizer.create('en-US-JennyNeural')`；進入練習時呼叫 `synthesizer.synthesize(question.text)` 朗讀題目；使用者以 Recorder 錄音作答，後端 `/api/grader/score-answer` 以 OpenAI 評分。

整合切入點：新頭像功能應作為一個獨立的「對話呈現區」元件（例如 `AvatarStage.svelte`），訂閱重構後 Synthesizer 所發出的 viseme/word 事件。由於 Message 模型已預留 avatar 欄位，可漸進式導入而不破壞既有頁面。

專案描述的釐清：目前 LLM 回覆以文字+（既有的）TTS 朗讀為主，尚無「LLM 串流回覆同時對嘴」的數位人。本功能即是在此基礎上，新增「會說話的臉」與「逐字字幕」。

4. 技術挑戰深入分析

即便來源端 (Azure) 已提供 viseme 與 word-boundary，要做到影片中那種「順、準、不延遲」的體驗，仍有以下挑戰需要逐一克服：

4.1 端到端延遲與串流分句

- **挑戰**：LLM 逐 token 產生 → 必須切成「語意完整、可合成」的句子才送 TTS，否則語音破碎；但等整段講完再合成又會有明顯延遲。
- **緩解**：沿用現有 partialSentenceBuffer 的「句末標點 + 逾時」分句策略；採「首句快出、後續排隊」的管線 (pipeline)，讓頭像盡早開口，後續句子在背景持續合成。

4.2 音訊、嘴型、字幕三軌時間對齊

- **挑戰**：viseme (60fps blendshape frame)、word-boundary (單字 tick)、與實際音訊播放，三者必須對齊到同一條時間軸。Azure 的 tick 為 100ns，需 $\div 10,000$ 換算為毫秒。
- **緩解**：以「音訊播放時鐘」為單一真實來源 (single source of truth)。每個 frame 攜帶 FrameIndex，於對應音訊區塊播放前套用；字幕以 requestAnimationFrame 每幀比對 currentTime 決定點亮哪個字。
- **已知風險**：Microsoft 社群回報，在特定 voice/輸出組合或含特殊字元時，audioOffset/duration 可能回傳 0 或些微偏移。須以實際使用的 voice 做驗證並加入容錯（如以字數平均估算為後備）。

4.3 瀏覽器渲染效能 (若採 3D 路線)

- **挑戰**：Three.js/WebGL 渲染 3D 頭像，加上每秒 60 次 blendshape 更新，需確保在中階裝置維持穩定 30–60fps，且不阻塞 Svelte 的反應式更新與錄音。
- **緩解**：頭像渲染與字幕分離為不同更新節奏；blendshape 套用在 rAF 迴圈中節流；頭像資產 (glb) 做減面與貼圖壓縮；行動裝置可降級為 2D viseme 圖序列或純字幕。

4.4 GPU 後端與串流 (若採 2D 神經頭像路線)

- **挑戰**：MuseTalk/Wav2Lip/Audio2Face 皆為伺服器端 PyTorch/CUDA，無法在瀏覽器 JS 執行；需要 GPU 主機，並以 WebRTC/MSE 把影格串回前端，增加部署、延遲與成本。

- **緩解**：僅在「必須照片級寫實」時才採用；以 MuseTalk（單步 latent inpainting、30fps+）為佳；建立 GPU 推論服務 + 串流通道，並設計無 GPU 時自動退回 3D/字幕的降級策略。

4.5 頭像資產與授權

- **挑戰**：3D 路線需要一個含 ARKit/Oculus viseme morph targets 的 rigged glTF 頭像；2D 路線需要一張可商用、形象合適的真人或虛擬人臉。
- **緩解**：3D 可用 Ready Player Me 產生免費可商用頭像（已含 ARKit morphs，與 TalkingHead/Azure 55-blendshape 對應）；2D 須留意肖像權與模型授權（Wav2Lip 為非商業，見第 5 章）。

5. 學術方法與開源方案評估

依使用者要求，以下方案皆有公開的 **GitHub 原始碼**，並區分兩大類：（A）3D blendshape/viseme 驅動（瀏覽器原生、即時、最貼合本專案）；（B）2D 神經像素生成（伺服器 GPU、照片級寫實）。所有事實已於 2026-06-19 對照各 repo 與 Microsoft 官方文件查核。

5.1 方法 A (首選) : Viseme/Blendshape 驅動 3D 頭像

研究基礎：視位（viseme）對嘴與 blendshape 臉部動畫為電腦動畫長期成熟之方法（如 JALI、Oculus LipSync、ARKit 表情基底）。在本專案中以開源函式庫落地：

開源實作：[met4citizen/TalkingHead](https://github.com/met4citizen/TalkingHead)（MIT 授權，約 1.4k★）

- 純 JavaScript，瀏覽器端以 Three.js/WebGL 即時渲染 Ready Player Me / glTF (.glb) 全身或半身數位人。
- 原生支援 Microsoft Azure Speech SDK；可直接吃外部 viseme 陣列（visemes / vtimes / vdurations）與 `speakAudio()` 動畫模板。
- 與 Azure `visemeReceived` 的 ARKit 55-通道 blendshape（60fps）對應乾淨。

優點：無需 GPU 後端、與現有 Azure TTS 串接最自然、延遲低、可商用（MIT）、行動裝置可降級。

限制：為「風格化 3D 頭像」而非真人影片；需準備符合 blendshape 規格的 rigged 頭像；為單一作者維護之專案（無正式 SLA）。

5.2 方法 B (備選) : MuseTalk 即時 2D 神經對嘴 (照片級寫實)

論文：MuseTalk: Real-Time High Quality Lip Synchronization with Latent Space Inpainting（arXiv 2410.10122, 2024；騰訊音樂 Lyra Lab）

開源實作：[TMElyralab/MuseTalk](https://github.com/TMElyralab/MuseTalk)（MIT 授權）

- 以 VAE 將臉編碼至潛在空間，對嘴部區域做單步 inpainting（非擴散多步迴圈），故能達 30fps+（256×256，V100），並宣稱啟動延遲極低。

- 輸入為真人影片/影像 + 音訊，輸出對嘴影片；需 NVIDIA GPU 後端，並以 WebRTC 串流回瀏覽器。

優點：寫實度最高、MIT 可商用、為目前少數真正「即時」的 2D 方案。

限制：需 GPU 主機與串流基礎設施、256² 嘴部區域、即時數據基於資料中心級 GPU；屬「編輯既有臉部影片」而非驅動 3D rig。

5.3 方法 C (備選) : Wav2Lip 基準 / NVIDIA Audio2Face 3D

C-1 Wav2Lip — [Rudrabha/Wav2Lip](#) (ACM MM 2020, 約 12k★)

- 經典 2D 對嘴基準，社群資源極豐、易重現；但上游為離線批次、非即時，輸出解析度低（96px 嘴部）。
- 授權警示：**原 repo 僅授權研究/個人非商業用途，不可直接商用（商用須洽 Sync Labs）。若產品為商業營運，建議排除。

C-2 NVIDIA Audio2Face — [NVIDIA/Audio2Face-3D](#) (2025 開源, SDK 多為 MIT)

- 音訊 → 3D 臉部 blendshape，原生即時 (>60fps)，支援串流音訊；可驅動 Unreal/Maya 角色。
- 限制：**需 NVIDIA GPU 後端、整合 footprint 重 (Omniverse 生態)、模型權重採 NVIDIA 自有授權（與 MIT 程式碼分離，需個別確認）。對本專案偏重。

5.4 三方案綜合比較

方案	類型	即時	瀏覽器原生	GPU 後端	授權	最適情境
TalkingHead.js (A)	3D viseme	是	是	不需	MIT	與 Azure TTS 串接、低成本、可商用（首選）
MuseTalk (B)	2D 神經	是	否	需	MIT	需照片級寫實且可投入 GPU
Wav2Lip (C-1)	2D 神經	否	否	需	非商業	研究/原型基準（商用排除）
Audio2Face (C-2)	3D 神經	是	否	需(NV)	MIT*	高品質 3D、已有 NVIDIA/Unreal 生態

**Audio2Face* 程式碼多為 *MIT*，但模型權重採 *NVIDIA* 模型授權，需個別確認。

選型建議

預設採方法 A（TalkingHead.js + Azure viseme）：最符合「容易實現、有開源、與現有 Azure 架構契合」三項要求，且能完整覆蓋對嘴 + 逐字字幕。

若教學需求明確要求「真人臉」，再以方法 B（MuseTalk）建置 GPU 服務作為進階形象；方法 C 作為基準對照或備援。

6. 字幕逐字標示 (單字級對齊) 方案

「目前唸到哪個字就標示哪個字」需要每個單字的音訊時間區間。本專案有兩條路：

6.1 首選：Azure WordBoundary 事件 (零額外基礎設施)

Azure TTS 合成時發出 `SpeechSynthesisWordBoundaryEventArgs`，含：

- `audioOffset` (單字起始, tick)、`duration` (單字長度, tick) → 決定「何時點亮」。
 $\text{tick} \div 10,000 = \text{毫秒}$ 。
- `text`、`wordLength`、`textOffset` (單字在原文中的字元位移) → 決定「點亮哪一段字元」。
- `boundaryType` (word / punctuation / sentence) → 可區分標點與句界。

做法：合成前保留原句字串；播放時以 `rAF` 迴圈比對 `audio.currentTime (ms)` 落在哪個單字的 `[offset, offset+duration]` 區間，套用高亮樣式。瀏覽器 JS SDK 即可取得此事件，無需後端模型。

6.2 備選：強制對齊 (僅在對「既有音檔」對齊時才需要)

工具	GitHub	授權	適用/限制
WhisperX	m-bain/whisperX	BSD-4	Whisper ASR + wav2vec2 音素級對齊，準確；離線、需 GPU。適合對非 Azure 既有音檔。
Montreal Forced Aligner	MontrealCorpusTools/MFA	MIT	Kaldi 強制對齊，成熟多語；離線 CLI、設定較重，不可瀏覽器部署。
Gentle	strob/gentle	MIT 類	已大致停止維護，不建議用於新專案。

結論

既然語音由 Azure 「即時產生」，直接用 `WordBoundary` 即可，毋須導入 `WhisperX/MFA`。後者只在未來要對「外部既有音檔」做字幕時才補上。

7. 推薦整合架構與資料流

7.1 端到端資料流 (首選方案 A)

4. 使用者送出訊息 → 後端呼叫 OpenAI，以串流方式回傳 token。
5. 前端/合成層累積 token，沿用現有 partialSentenceBuffer 邏輯做「逐句」切分。
6. 每一句以 SSML 包覆並加入 `<mstts:viseme type="FacialExpression"/>` 後送 Azure TTS (要求 blendshape 動畫輸出)。
7. 重構後的 Synthesizer 掛載三組回呼：`synthesizing` (音訊)、`visemeReceived` (55-blendshape@60fps)、`wordBoundary` (單字 tick)。
8. 音訊以可定位的播放器播放，並暴露 `currentTime` 作為唯一時鐘。
9. `AvatarStage.svelte` 依 `FrameIndex` 在對應音訊區塊前套用 blendshape 至 TalkingHead 頭像；`Subtitle.svelte` 依 `currentTime` 逐字高亮。

7.2 建議新增/重構的程式元件

元件	類型	職責
synthesizer.ts (重構)	既有	改為可定位播放 + 發出 viseme/word 事件；SSML 注入 mstts:viseme；保留逐句切分。
AvatarStage.svelte	新增	Three.js 容器，載入 glTF 頭像，訂閱 viseme 事件、依播放時鐘套用 blendshape。
Subtitle.svelte	新增	接收原句 + word-boundary 陣列，rAF 逐字高亮；含容錯 (offset=0 時以字數平均估算)。
TalkingHead 整合模組	新增	封裝 met4citizen/TalkingHead，提供 setBlendshapes()/speakAudio() 介面。
avatarStore (Svelte store)	新增	集中管理頭像狀態 (說話中/閒置)、目前句、目前字索引，供多頁面重用。
(選用) GPU 對嘴服務	新增	僅方法 B/C 需要：FastAPI/Triton + WebRTC 串流真人對嘴影像。

7.3 SSML 範例 (取得 blendshape 與 word boundary)


```
< speak xmlns:mstts="..." xml:lang="en-US">< voice name="en-US-JennyNeural"><mstts:viseme type="FacialExpression"/>Hello, how are you today?</voice></speak>
```

說明：加入 mstts:viseme 後，visemeReceived 事件的 e.animation 會帶回 ARKit 55-blendshape 的 JSON（{FrameIndex, BlendShapes:[...]，每幀 55 個 0–1 數值，60fps）；WordBoundary 事件不需額外宣告即會發出。

8. 需額外新增的功能 (分析過程中發現)

除頭像與字幕本身外，要讓功能完整、可用、可維護，建議一併納入以下項目：

10. **Synthesizer 播放時鐘重構 (必要)**：如 3.2 所述，這是 viseme/字幕能精準對齊的前提，屬阻塞性前置工作。
11. **LLM 串流回覆通道**：目前 LLM 用於評分 (非串流對話)。本功能需要一條「LLM 逐 token 串流 → 即時 TTS」的對話通道 (SSE/WebSocket)。
12. **打斷 (barge-in) 處理**：使用者中途開口時，需中止頭像說話、清空字幕、停止合成。現有 Synthesizer.interrupt() 可擴充為同時通知頭像/字幕重置。
13. **頭像資產管線**：Ready Player Me 頭像取得、減面、ARKit morph 驗證、載入快取 (避免每次對話重載 glb)。
14. **降級策略 (progressive enhancement)**：依裝置能力自動退級：3D 頭像 → 2D viseme 圖序列 → 純逐字字幕；確保低階裝置與無 WebGL 環境仍可用。
15. **音訊與字幕快取/回放**：呼應專案「保存對話音頻與逐字稿」之既有目標，將每則頭像回覆的音檔 + word-boundary 時間軸一併保存，供事後逐字回放 (與既有 AudioPlayer 整合)。
16. **無障礙與設定**：提供「關閉頭像/僅字幕」「字幕字級」「播放速度」等設定；字幕需符合可讀對比與 ARIA。
17. **效能監測與容錯**：監測 fps 與 A/V 偏移；對 Azure offset=0 邊界情況加入後備估算與記錄。

9. 分階段實作路線圖

建議採漸進式交付，每階段皆可獨立驗證，風險由低到高：

階段	目標	主要工作	驗收標準
P0	訊號打通 (PoC)	重構 Synthesizer 暴露播放時鐘；掛載 visemeReceived / wordBoundary；console 印出 blendshape 幀與單字 tick。	能穩定取得 55-blendshape 幀與單字時間，且與音訊對齊誤差可量測。
P1	逐字字幕	新增 Subtitle.svelte，依 WordBoundary 即時高亮目前單字（先不接頭像）。	朗讀任一句時，字幕逐字點亮且與語音同步。
P2	3D 頭像對嘴	整合 TalkingHead.js + Ready Player Me 頭像；以 viseme 串流驅動嘴型。	頭像嘴型與語音對齊，畫面 $\geq 30\text{fps}$ 。
P3	對話串流整合	建立 LLM 串流通道 + 逐句管線；加入 barge-in 中止與重置。	完整一輪「使用者問→頭像邊說邊對嘴邊高亮」順暢無明顯延遲。
P4	保存/回放與降級	保存音檔 + 時間軸；事後逐字回放；裝置降級策略；設定與無障礙。	對話可回放逐字稿；低階裝置自動降級且不報錯。
P5 (選用)	照片級寫實	建置 MuseTalk GPU 服務 + WebRTC 串流，作為進階頭像形象。	真人頭像即時對嘴，並能在無 GPU 時自動退回 P2。

最小可行範圍 (MVP)

若要最快看到影片中的效果，**P0 + P1 + P2 即構成 MVP**：在一個示範頁面上，讓 3D 頭像朗讀一段固定英文、嘴型對齊、字幕逐字高亮。此範圍完全不需 GPU 後端，全部以現有 Azure 相依達成。

10. 風險與緩解

風險	說明	緩解
Azure word/viseme 邊界異常	特定 voice 或含特殊字元時 offset/duration 可能為 0 或偏移。	以實際 voice 驗證；加入字數平均後備估算；記錄並監測偏移。
A/V 同步漂移	長句或網路抖動下，嘴型/字幕可能與音訊累積偏移。	以音訊 currentTime 為單一時鐘；逐句重新校時；句界重置。
瀏覽器效能不足	低階/行動裝置 WebGL + 60fps blendshape 卡頓。	渲染節流與減面；降級為 2D/純字幕；偵測 WebGL 能力。
授權合規	Wav2Lip 非商業；Audio2Face 權重、肖像權、頭像資產授權。	商用優先選 MIT (TalkingHead / MuseTalk)；用 Ready Player Me 可商用頭像。
GPU 後端成本 (B/C)	即時 2D 神經對嘴需常駐 GPU，成本與維運高。	僅在必要時導入；無 GPU 自動降級；評估雲端 GPU 彈性擴縮。
範圍蔓延	頭像功能易擴張（表情、情緒、全身動作）。	嚴守 MVP (P0-P2) 先行；後續功能以迭代加入。

11. 結論與建議

影片所展示的「邊說、邊對嘴、邊逐字標示」體驗，對本專案而言**門檻低於一般直覺**——因為達成它所需的兩個最難訊號（嘴型與單字時間）已由現有的 Azure TTS 在合成當下原生提供。真正的工程重點不在「生成對齊資訊」，而在「把這些資訊乾淨地接到一個瀏覽器端的頭像與字幕時間軸上」。

明確建議：以

- 方法 A (met4citizen/TalkingHead + Azure visemeReceived) 為主線實作對嘴；
- Azure WordBoundary 實作逐字字幕；
- 先交付 P0-P2 的 MVP (無需 GPU)，驗證體驗後，再視「是否需要真人臉」決定是否投入方法 B (MuseTalk) 的 GPU 路線。

此路線在「容易實現、具開源、與現有架構契合、授權可商用」四項要求上取得最佳平衡，並保留未來升級至照片級寫實數位人的空間。

附錄 A：開源專案與授權清單

專案	GitHub	授權	即時	備註
TalkingHead	met4citizen/TalkingHead	MIT	是	3D glTF，瀏覽器原生，支援 Azure viseme（首選）
MuseTalk	TMElyralab/MuseTalk	MIT	是	2D 潛在 inpainting，30fps+，需 GPU
Wav2Lip	Rudrabha/Wav2Lip	非商業	否	ACM MM 2020 基準，商用須授權
SadTalker	OpenTalker/SadTalker	Apache-2.0	否	CVPR 2023，單圖→影片，離線
Audio2Face	NVIDIA/Audio2Face-3D	MIT*	是	2025 開源，3D blendshape，需 NVIDIA GPU
GeneFace++	yerfor/GeneFacePlusPlus	見 repo	是	NeRF，需逐人訓練
LivePortrait	KwaiVGI/LivePortrait	MIT(含依賴限制)	近即時	InsightFace 依賴為非商業
WhisperX	m-bain/whisperX	BSD-4	否	單字級強制對齊（備選）
Montreal Forced Aligner	MontrealCorpusTools/MFA	MIT	否	Kaldi 強制對齊（備選）

* 程式碼多為 MIT，模型權重採 NVIDIA 模型授權；星數與授權以查核日 2026-06-19 為準，使用前請再次確認。

附錄 B：Azure Speech SDK 關鍵事件速查

事件	關鍵欄位	用途
visemeReceived	visemeId(0-21)；e.animation：{FrameIndex, BlendShapes[][55]} 60fps（需 SSML mstts:viseme）	驅動 3D 頭像對嘴（ARKit 55-blendshape）
wordBoundary	audioOffset, duration	字幕逐字高亮（何時亮、亮哪段字元）

事件	關鍵欄位	用途
	(tick=100ns , ÷10000=ms) , text, wordLength, textOffset, boundaryType	
synthesizing / synthesisCompleted	audioData (分塊)	取得音訊資料與合成完成通知 (現有已使用)

資料來源：Microsoft Learn (viseme 文件更新於 2026-02-25 ； JS API 參考更新於 2025-09-23) 。瀏覽器 JS SDK (microsoft-cognitiveservices-speech-sdk) 皆可取得上述事件。